CO4: Analyze different hashing techniques and collision resolution strategies.

4.1 HASHING

Hashing is a technique of mapping a large set of arbitrary data to tabular indexes using a hash function. It is a method for representing dictionaries for large datasets. It allows lookups, updating and retrieval operation to occur in a constant time i.e. O(1).

Why Hashing is Needed?

After storing a large amount of data, we need to perform various operations on these data. Lookups are inevitable for the datasets. Linear search and binary search perform lookups/search with time complexity of O(n) and O(log n) respectively. As the size of the dataset increases, these complexities also become significantly high which is not acceptable. We need a technique that does not depend on the size of data. Hashing allows lookups to occur in constant time i.e. O(1).

4.1.1 Hash Function

A hash function is used for mapping each element of a dataset to indexes in the table. The Hash table data structure stores elements in key-value pairs where:

- **Key:** Unique integer that is used for indexing the values.
- **Value:** Data that are associated with keys.



Fig. 4.1: Key and value in hash table

In a hash table, a new index is processed using the keys. And, the element corresponding to that key is stored in the index. This process is called hashing. Let, k be a key and h(x) be a hash function. Here, h(k) will give us a new index to store the element linked with k.

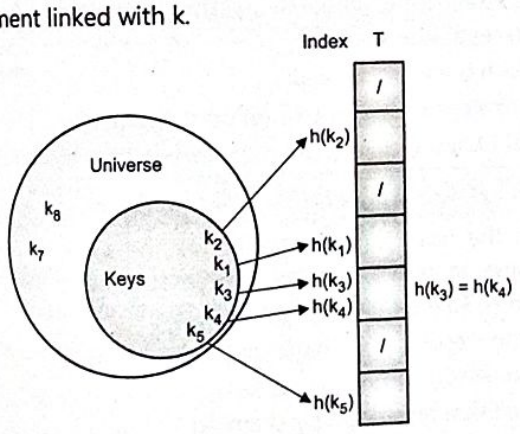


Fig. 4.2: Hash table representation

4.1.2 Scatter Tables

The separately chained hash table described in the preceding section is essentially a linked-list implementation. We have seen both linked-list and array-based implementations for all of the data structures considered so far and hash tables are no exception. Array-based hash tables are called scatter tables.

The essential idea behind a scatter table is that all of the information is stored within a fixed size array. Hashing is used to identify the position where an item should be stored. When a collision occurs, the colliding item is stored somewhere else in the array.

One of the motivations for using scatter tables can be seen by considering again the linked-list hash table shown in Fig. 4.3. Since most of the linked lists are empty, much of the array is unused. At the same time, for each item that is added to the table, dynamic memory is consumed. Why not simply store the data in the unused array positions.

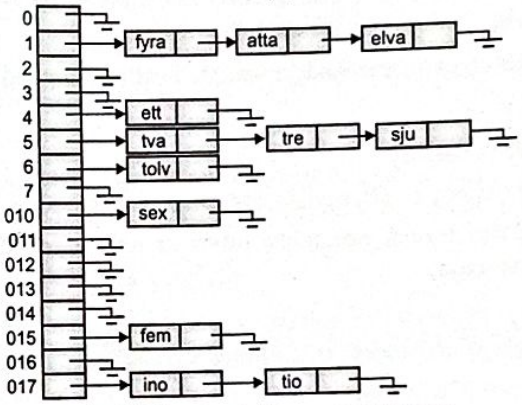


Fig. 4.3: Scatter table representation

4.2 BASIC OPERATIONS

A Hash Table is a data structure that maps keys to values using a hash function to compute an index into an array, from which the desired value can be found.

Below are the basic operations performed on a hash table:

1. Insert (Put)

Definition: Adds a new key-value pair to the hash table.

Process:

- Compute the hash code using the key.
- Map it to an index using the hash function.
- If no collision, insert directly at that index.
- If a collision occurs, resolve it (e.g., via chaining or open addressing).

Example:

```
table[hash("apple")] = "fruit";8
```

2. Search (Get or Lookup)

Definition: Retrieves the value associated with a given key.

Process:

- Hash the key to find the index.
- Access the index directly.
- If collision resolution is used, traverse the chain or probe sequence.

Example:

```
value = table[hash("apple")]; // Returns "fruit"
```

3. Delete (Remove)

Definition: Removes a key-value pair from the table.

Process:

- Compute the hash index of the key.
- Locate the key (using chaining or probing if necessary).
- Remove the key-value pair.

Example:

```
remove("apple"); // Deletes the entry from table
```

4. Update

Definition: Updates the value associated with an existing key.

Process:

- Hash the key.
- If the key exists, update the value at that index.
- If not found, optionally insert as new (depends on use-case).

Example:

```
table[hash("apple")] = "red fruit"; // Updates value
```

5. Traverse (Optional)

Definition: Visit and process each key-value pair.

Use Case: Displaying all entries or exporting data.

Example Logic:

```
for each bucket in table:
  for each node in bucket:
    print(node.key, node.value)
```

Table 4.1 : Summary Table

Operation	Description
Insert	Add a new key-value pair
Search	Retrieve value using a key
Delete	Remove a key and its associated value
Update	Change value for an existing key
Traverse	(Optional) Visit all elements

4.3 BUCKET

Bucket in Hash Tables

In hash tables, a bucket refers to a storage location (container) at a particular index of the hash table array where one or more key-value pairs are stored. Buckets are primarily used to handle collisions, which occur when multiple keys hash to the same index.

Purpose of Buckets

When two or more keys produce the same hash index (collision), a single index must store multiple entries. Buckets provide a way to store these multiple entries at the same index in the hash table.

Types of Bucket Implementations

1. Separate Chaining:

In this approach, each bucket is typically a linked list (or dynamic structure like a vector). All key-value pairs that hash to the same index are inserted into the same bucket.

Example:

```
Table[3] → [ ("apple", 1) → ("grape", 2) → ("melon", 3) ]
```

2. Open Addressing:

In this method, the hash table itself does not use extra buckets. Instead, if a collision occurs, the algorithm searches for the next available slot using techniques like linear probing or quadratic probing. Each slot (bucket) holds only one key-value pair.

Example of Bucket Use (Separate Chaining)

Assume a hash table of size 5 and the following hash results:

- hash("apple") % 5 = 2
- hash("grape") % 5 = 2

Both keys map to index 2. In separate chaining, index 2 becomes a bucket that stores both entries:

```
Table[2] → [ ("apple", value1), ("grape", value2) ]
```

Summary

- A bucket is a logical container used to store key-value pairs at a particular index in a hash table.
- Buckets are essential in collision resolution, especially in separate chaining.
- Each bucket may hold:
  - ➢ One key-value pair (in open addressing)
  - ➢ Multiple key-value pairs (in separate chaining)

4.4 HASH COLLISION

When the hash function generates the same index for multiple keys, there will be a conflict (what value to be stored in that index). This is called a hash collision.

We can resolve the hash collision using one of the following techniques:

- Collision resolution by chaining
- Open Addressing: Linear/Quadratic Probing and Double Hashing

4.4.1 Probe

Definition:

A probe refers to the act of searching or checking a position in the hash table to insert, search, or delete a key. Probing is used in open addressing methods to resolve collisions by examining alternative locations in the table.

Types of Probing:

Linear Probing: Checks the next index sequentially.

```
index = (hash(key) + i) % table_size
```

Quadratic Probing: Uses quadratic increments.

```
index = (hash(key) + i²) % table_size
```

Double Hashing: Uses a second hash function to calculate the probe step size.

4.4.2 Synonym

Definition:

Synonyms are different keys that produce the same hash value (i.e., they map to the same index in the hash table). When synonyms occur, it leads to a collision.

Example:

hash("cat") % 10 = 5

hash("act") % 10 = 5

Here, "cat" and "act" are synonyms.

4.4.3 Overflow

Definition:

Overflow in hashing refers to the condition where a calculated index in the hash table is already occupied, and there is no immediate space to store the new item. It typically occurs due to collisions and limited table size.

Handling Overflow:

- **Chaining (Open Hashing):** Store collided entries in a linked list.
- **Probing (Closed Hashing):** Search for alternative space in the table.

4.4.4 Open Hashing (Separate Chaining)

Definition:

Open Hashing, also known as separate chaining, is a collision resolution technique in which each index of the hash table contains a pointer to a linked list (or other structure). All items that hash to the same index are stored in this list.

Characteristics:

- Table size can be smaller than the number of keys.
- No clustering problem.
- Extra memory used for chains.

Example:

Table[3] → [key1 → key2 → key3]

4.4.5 Closed Hashing (Open Addressing)

Definition:

Closed Hashing, also known as open addressing, is a collision resolution technique where all elements are stored directly within the hash table. When a collision occurs, the algorithm probes the table to find the next available slot.

Types of Probing in Closed Hashing:

- Linear Probing
- Quadratic Probing
- Double Hashing

Characteristics:

- All keys stored in the hash table array itself.
- Load factor should be kept low for performance.
- Suffering from clustering if not handled properly.

4.4.6 Perfect Hash Function

Definition:

A perfect hash function is a hash function that produces no collisions for a given set of keys. That is, each key maps to a unique index in the hash table.

Characteristics:

- It is ideal for static datasets (where the set of keys is fixed).
- Used when fast, conflict-free lookups are required (e.g., compiler keyword tables).
- Requires pre-analysis of the data to design such a function.

Types:

Minimal Perfect Hash Function: A perfect hash function where the number of slots equals the number of keys, with no wasted space.

Example Use Case:

A list of programming language keywords (if, else, for, while, return) - a perfect hash function maps each uniquely to an index.

4.4.7 Load Density and Full Table, Load Factor and Rehashing

Definition:

Load density refers to the number of elements stored per bucket in a hash table, particularly in the context of open hashing (separate chaining).

Formula:

Load Density = Total number of records / Total number of buckets

Interpretation:

A higher load density means more records are stored per bucket, increasing the chance of longer chains, hence reducing performance.

Ideally, load density should be low to ensure faster access time.

4.4.8 Full Table

Definition:

A full table is a condition in which all slots in the hash table are occupied. This term is primarily relevant in closed hashing (open addressing), where all data is stored directly in the array.

Consequences:

- No further insertions can be performed without expanding the table.
- Causes infinite probing if insertion is attempted.
- Must trigger rehashing or resizing of the table.

Note: In open addressing, once the load factor approaches 1 (100% filled), the table is effectively full.

4.4.9 Load Factor

Definition:

Load factor (also called load ratio) is a measure that indicates how full a hash table is. It influences performance and guides when to resize or rehash the table.

Formula:

Load Factor (α) = Number of entries / Table size

Ideal Range:

- For open addressing, load factor should ideally be ≤ 0.7.
- For separate chaining, it can exceed 1.0 (since each bucket can store multiple entries).

Implications:

- A low load factor leads to faster access and fewer collisions.
- A high load factor increases collisions and degrades performance.

4.4.10 Rehashing

Definition:

Rehashing is the process of creating a larger hash table and re-inserting all existing keys using a new hash function or a modified table size.

Why Rehash?

- Occurs when the load factor exceeds a threshold.
- Prevents performance degradation due to collisions.
- Avoids problems caused by a full table in open addressing.

Process:

1. Create a new table of larger size (usually double).
2. Compute new hash values for existing keys.
3. Insert all keys into the new table.

Note: Rehashing is a costly operation, but it's necessary for maintaining amortized constant time performance for hash table operations.

4.5 PROPERTIES OF GOOD HASH FUNCTIONS

A good hash function may not prevent the coll... completely however it can reduce the number of collision...

Here, we will look into different methods to find a ... hash function.

1. **Division Method**

If k is a key and m is the size of the hash table, the ... function h() is calculated as:

$$h(k) = k \bmod m$$

For example, if the size of a hash table is 10 ... k = 112 then h(k) = 112 mod 10 = 2. The value of m mus... be the powers of 2. This is because the powers of 2 in bi... format are 10, 100, 1000, ... When we find k mod m, we ... always get the lower order p-bits.

if m = 22, k = 17,
then h(k) = 17 mod 22 = 10001 mod 100 = 01

if m = 23, k = 17,
then h(k) = 17 mod 22 = 10001 mod 100 = 001

if m = 24, k = 17,
then h(k) = 17 mod 22 = 10001 mod 100 = 0001

if m = 2p,
then h(k) = p lower bits of m

2. **Multiplication Method**

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

where,
kA mod 1 gives the fractional part kA,
$\lfloor \rfloor$ gives the floor value

A is any constant. The value of A lies between 0 and ... But, an optimal choice will be ≈ (√5-1)/2 suggested by Knut...

3. **Universal Hashing**

In Universal hashing, the hash function is chosen ... random independent of keys.

Python to Demonstrate Working

```
# Python program to demonstrate working of dictionary
(hashtable)


# Create a dictionary
ht = {}


# Add key-value pairs
ht[123] = 432
ht[12] = 2345
ht[15] = 5643
ht[3] = 321
```

```
# Remove a key
ht.pop(12)
# Print the dictionary
print(ht)
```

4.6 COLLISION RESOLUTION STRATEGIES

Hashing is a fundamental technique in data structures used for efficient data storage and retrieval. A hash function maps keys to positions in a hash table, allowing operations like insertion, deletion, and search to be performed in constant average time O(1). However, due to the possibility of collisions where different keys produce the same hash index collision resolution strategies are necessary. This section discusses in detail the main collision resolution strategies and advanced methods like extendible hashing.

A collision occurs when two or more keys are hashed to the same index. To resolve such conflicts, two main families of collision resolution strategies are used:

1. Open Addressing
2. Chaining (Closed Addressing / Separate Chaining)

4.6.1 Open Addressing

Definition:

In open addressing, all elements are stored directly in the hash table array itself. If a collision occurs, the algorithm probes (searches) for the next available slot based on a predefined sequence until an empty location is found.

Types of Probing Methods:

1. **Linear Probing:**

Increments the index by 1 each time until an empty slot is found.

Formula:

$$index = (hash(key) + i) \% table_size$$

where $i = 0, 1, 2, \ldots$

Advantages:

- Simple to implement.
- Good cache performance due to sequential access.

Disadvantage:

- Prone to primary clustering (keys cluster together, increasing search time).

2. **Quadratic Probing:**

Probes using quadratic increments to reduce clustering.

Formula:

$$index = (hash(key) + c1*i + c2*i^2) \% table_size$$

Advantage:

- Reduces primary clustering.

Disadvantage:

- Still susceptible to secondary clustering (same hash values lead to same probe sequences).

3. **Double Hashing:**

Uses a second hash function to determine the step size for probing.

Formula:

$$index = (hash1(key) + i * hash2(key)) \% table_size$$

Advantage:

- Reduces clustering more effectively than other methods.

Disadvantages:

- More complex to implement.
- Requires a good second hash function.

4.6.2 Chaining (Closed Addressing or Separate Chaining)

Definition:

In chaining, each index in the hash table holds a pointer to a linked list (or any dynamic structure) that stores all keys hashing to that index.

Structure:

Hash Table:

```
Index 0 → [Key1 → Key2 → Key3]
Index 1 → [Key4]
Index 2 → [Empty]
...
```

Advantages:

- Allows more keys than the table size.
- Load factor can exceed 1.
- Simplifies deletion (just delete from the linked list).

Disadvantages:

- Requires additional memory for pointers and dynamic storage.
- Cache performance is lower than open addressing.
- Lookup time may degrade to O(n) in the worst case.

4.7 HASH TABLE OVERFLOW

4.7.1 Overflow in Open Addressing

When the hash table becomes full or nearly full, it becomes difficult to find an empty slot for insertion using open addressing. As more probing is required, performance degrades significantly.

Conditions:

- Load factor (α) approaches 1.
- All probe sequences fail to find an empty slot.

Handling:

- Rehashing to a larger table.
- Doubling the table size and reinserting all elements.

4.7.2 Overflow in Chaining

Overflow in chaining occurs when the chains (linked lists) at particular indices become excessively long.

Consequences:

- Increased memory usage.
- Lookup time increases, especially in the worst case.

Handling:

- Use better hash functions to distribute keys more evenly.
- Use dynamic resizing and rehashing strategies.
- Use self-balancing trees (like AVL trees) or dynamic arrays instead of linked lists in buckets.

4.7.3 Extendible Hashing

Definition:

Extendible Hashing is a dynamic hashing technique that uses a directory of pointers to buckets, allowing the hash table to grow or shrink as needed.

How it Works?

- Each key is hashed to a binary value.
- A prefix of this binary value is used to index into a directory.
- The directory points to buckets that hold the actual records.
- When a bucket overflows, only that bucket is split, and the directory is updated accordingly.

Advantages:

- Provides efficient dynamic resizing.
- Avoids global rehashing.
- Lookup, insertion, and deletion remain efficient even under high load.

Disadvantages:

- More complex than standard hash tables.
- Uses extra memory for directory management.

Example:

Suppose the directory has 2 bits: 00, 01, 10, 11

hash("apple") → 100101 → uses 10 → goes to bucket 10

On collision, if bucket 10 overflows, it is split, and the directory may be expanded

4.7.4 Closed Addressing vs Open Addressing

Table 4.2

Feature	Open Addressing	Closed Addressing (Chaining)
Storage location	Inside the hash table	External structures (linked list, etc.)
Load factor limit	≤ 1	> 1 allowed
Collision handling	Probing for new slots	Inserting into a bucket (list)
Memory usage	More space-efficient	Uses extra memory for chains
Cache performance	Better (sequential access)	Worse (pointer access)
Deletion complexity	Complex (requires tombstones)	Simple (delete from list)
Performance degradation	Severe when full	Smooth degradation

4.7.5 Advanced Notes and Best Practices

1. **Choosing Hash Function**
   - Uniform distribution is key.
   - Avoid clustering.
   - Use prime table sizes when possible.
2. **Load Factor Management**
   - In open addressing: keep load factor < 0.7
   - In chaining: acceptable to go beyond 1.0 but mo chain lengths
3. **Hybrid Structures**
   - Some modern systems use hybrid hashing chaining with trees or vectors).
   - Java's HashMap (Java 8+) converts long chains balanced trees for efficiency.

CASE STUDY

Dictionary Application using Hash Tables, Descripti Implement a dictionary where words and meanings stored and retrieved using hashing with colli resolution

Problem Statement

Design and implement a dictionary application supports:

- Insertion of a word and its meaning
- Searching for the meaning of a given word
- Deletion of a word

The solution must ensure fast access time, even wit large number of entries.

Real-World Context

In digital dictionaries, thesauruses, and language to (like Oxford or WordWeb), users expect instantaneo lookups for words and their meanings. Traditional lin search methods become inefficient at scale. Hashing provid constant-time average complexity, making it ideal for su applications.

Data Structures Used

- Hash Table using an array of linked lists (separa chaining)
- Node Structure: Each node contains a word, meaning, and a pointer to the next node

Why Hash Table with Collision Resolution?

- Hashing allows O(1) average time for insert / search delete.
- Collisions (two words hashing to the same index) a inevitable.
- Separate chaining (linked lists at each hash inde provides an efficient way to handle collisions

Hash Function

We use a simple polynomial rolling hash:

```
TABLE_SIZE = 100

class Node:
  def __init__(self, word, meaning):
    self.word = word
    self.meaning = meaning
    self.next = None

class Dictionary:
  def __init__(self):
    self.table = [None] * TABLE_SIZE

  def hash_function(self, word):
    hash_value = 0
    for ch in word:
      hash_value = (hash_value * 31 + ord(ch)) % TABLE_SIZE
    return hash_value

  def insert(self, word, meaning):
    index = self.hash_function(word)
    new_node = Node(word, meaning)
    new_node.next = self.table[index]
    self.table[index] = new_node

  def search(self, word):
    index = self.hash_function(word)
    current = self.table[index]
    while current:
      if current.word == word:
        return current.meaning
      current = current.next
    return "Word not found."

  def remove(self, word):
    index = self.hash_function(word)
    current = self.table[index]
    prev = None

    while current:
      if current.word == word:
        if prev:
          prev.next = current.next
        else:
          self.table[index] = current.next
        print("Word deleted successfully.")
        return
      prev = current
      current = current.next
    print("Word not found.")

  def display(self):
    for i in range(TABLE_SIZE):
      current = self.table[i]
      if current:
        print(f"Index {i}: ", end="")
        while current:
          print(f"[{current.word}: {current.meaning}] -> ",
end="")
          current = current.next
        print("NULL")

# --- Main ---
if __name__ == "__main__":
  dict_obj = Dictionary()
  dict_obj.insert("apple", "A fruit that keeps doctors away.")
  dict_obj.insert("banana", "A yellow fruit.")
  dict_obj.insert("cat", "A small domesticated carnivorous
mammal.")
  dict_obj.insert("dog", "A faithful animal.")

  print("Searching for 'cat':", dict_obj.search("cat"))
  print("Searching for 'book':", dict_obj.search("book"))

  dict_obj.remove("banana")
  dict_obj.remove("book")

  print("\nCurrent Dictionary:")
  dict_obj.display()
```

Complexity Analysis

Operation	Time Complexity (Average)	Time Complexity (Worst Case)
Insert	O(1)	O(n) (all in one bucket)
Search	O(1)	O(n)
Delete	O(1)	O(n)

Enhancements

- Use dynamic resizing when load factor > threshold.
- Use open addressing (e.g., linear probing) instead of chaining.
- Use trie for prefix-based search.
- Support case-insensitive lookups.

Takeaways

- Hash Tables provide excellent average-case performance for dictionary-like applications.
- Separate chaining handles collisions well, and is simple to implement.
- The efficiency of the hash function significantly affects overall performance.

EXERCISE

1. Define a hash table. What is the purpose of using a hash function in a hash table? Provide an example of a simple hash function.
2. List and explain the basic operations that can be performed on a hash table. Illustrate any one operation with a brief example.
3. What is a bucket in the context of a hash table? How are buckets used in separate chaining?
4. Explain the terms collision, synonym, and overflow in the context of hashing. Provide examples to support your explanation.
5. What is probing in open addressing? Describe and differentiate between linear probing, quadratic probing, and double hashing.
6. Differentiate between open hashing and closed hashing. State one advantage and one disadvantage of each method.

7. What is a perfect hash function? How is it different from an ordinary hash function? Give a real-world use case where a perfect hash function is desirable.
8. Define and explain the following terms with formulas where applicable:
   (a) Load Density
   (b) Load Factor
   (c) Full Table
9. What is rehashing in hash tables? When is it necessary, and how is it performed? Discuss its impact on performance.
10. Explain the following collision resolution strategies with diagrams or examples:
    (a) Open Addressing
    (b) Separate Chaining

    Also, describe how overflow is handled in each case.